TECHNICAL REQUIREMENTS DOCUMENT

AICTE Approval Workflow Automation, Compliance Monitoring and Alert Management System

Problem Statement ID: 26262

Version: 2.0 **Updated:** 29 October 2026

Updated implementation context
Four-person hackathon team: two frontend developers, one backend developer (Python), and one integration developer who connects backend features to frontend screens. Work starts from a blank project. Demo/submission deadline is near, so the build prioritises a credible end-to-end prototype.

Core design decision
Next.js frontend + Python FastAPI backend + Supabase-hosted PostgreSQL. Use one API service with clear modules—not microservices. Start with list/detail APIs and seeded demo data so frontend integration can proceed immediately.

Classification key: [R] required by the supplied problem statement [D] proposed design [A] assumption [V] validate against the applicable APH / authorised AICTE rules before production.

Document guide

Section	Contents
1	Executive recommendation and revised team model
2	Scope, assumptions, and guardrails
3	Technology stack
4	Architecture and request flow
5	Modules and ownership
6	Data model
7	API contracts
8	Workflow and state transitions
9	Auth, permissions, audit, and data protection
10	Deadlines, alerts, and escalation
11	Frontend-to-backend integration contract
12	Environment setup and deployment
13	Build sequence: P0 / P1 / P2
14	Four-person work plan and Git workflow
15	Testing and acceptance criteria
16	Risks and decisions to validate
17	Demo checklist and suggested repository structure

This is a proposed technical design, not an official AICTE API specification. Endpoint names, schema names, status labels, and sample data are implementation proposals. Statutory routes, timelines, permissions, and decision rules must be validated against the applicable Approval Process Handbook (APH) and authorised instructions before real use.

1. Executive recommendation

Build a small, modular web application with a separately deployable FastAPI backend. This reflects the team's actual split and lets the backend developer work in Python while frontend developers work in TypeScript. The integration teammate should be able to connect screens to stable API contracts without waiting for every workflow rule to be complete.

Layer	Recommended choice	Purpose
Frontend	Next.js + TypeScript	Screens, navigation, tables, forms, role-oriented views.
UI	Tailwind CSS + shadcn/ui	Consistent reusable components and rapid styling.
Backend API	Python + FastAPI + Pydantic	HTTP endpoints, validation, workflow rules, permissions, audit events.
Database	Supabase-hosted PostgreSQL	Persistent applications, workflow history, EVC, compliance, documents, alerts.
DB access	SQLAlchemy + Psycopg	One consistent backend database access path and transaction handling.
Charts	Recharts (frontend)	Pendency, ageing, stage distribution, workload and compliance visualisations.

Layer	Recommended choice	Purpose
Tests	Pytest + FastAPI TestClient	Endpoint, validation, workflow and regression checks.
Local API server	Uvicorn	Run FastAPI locally during development.
Source control	GitHub	Shared code, PRs, issue tracking and stable demo checkpoints.

Do not introduce Redis, Celery, Kubernetes, microservices, a separate analytics database, or an AI/LLM service for the hackathon unless a demonstrated requirement forces the choice. A simple scheduled job or manual demo-triggered alert generation is sufficient for P0; dependable background scheduling belongs in P1/P2.

Critical scope boundary: the system supports authorised human review. It must not autonomously make statutory approval/rejection decisions or bypass a required scrutiny, inspection, committee, or verification step.

2. Scope, assumptions, and guardrails

Problem capabilities to demonstrate

- Configurable application routes, including the supplied route families: Only Scrutiny, Scrutiny + EVC, EVC + SAC/SHC, Only EVC, and other APH-defined combinations.
- Stage status, assignment, ageing, due dates, and authorised return/revert with a reason and requested clarification items.
- EVC queue, scheduling/completion status, deadline reminders, and escalation visibility.
- Time-bound undertakings, evidence submission/review, lease/document validity and expiry tracking.
- Dashboard filters and summaries for region, stage, institution, officer/committee, pending age, and workload.
- Configurable notifications, audit trail, reporting, and role-based access.

Assumptions and non-goals

- [A] The hackathon demo can use synthetic institutions, applications, officers, dates, evidence metadata, and event history.
- [A] No existing AICTE system API credentials or approved integration specification has been supplied. External integrations are therefore not assumed.
- [D] Route definitions and deadline thresholds are stored/configurable rather than scattered as hard-coded assumptions.
- [D] File upload may be simulated in P0; if enabled, use private storage and metadata records, not public file URLs.
- Out of scope for P0: production-grade identity integration, legal/statutory rule certification, automated inspection decisions, and full integration with live AICTE systems.

Deadline dates, stage names, and routes in seed data are illustrative unless explicitly verified. Label the demo data as synthetic and do not present sample metrics as official AICTE figures.

3. Technology stack and conventions

Concern	Decision / convention
API	FastAPI, Python type hints, Pydantic request/response schemas, versioned prefix /api/v1.
Database	PostgreSQL on Supabase; SQLAlchemy models and transactions; Psycopg driver.
Migrations	Use Alembic once schema changes begin to matter; for a very short P0, keep a documented initial schema and migration plan.
Configuration	Environment variables via .env locally; commit .env.example only. Never commit secrets.
Validation	Pydantic for request shape; service layer for business rules; database constraints for invariants.
Errors	Consistent HTTP status codes and JSON error bodies; no stack traces or secrets returned to browser.
Dates	Store timestamps in UTC; render local time in UI. Store date-only expiry/deadline fields as DATE where time-of-day is not meaningful.
Identifiers	Use stable UUIDs internally; expose human-friendly application numbers separately.
Naming	snake_case in Python/JSON unless frontend team agrees otherwise; document all enums and fields.

Use SQLAlchemy/Psycopg as the single database access path from FastAPI. Supabase may provide managed PostgreSQL and, if selected later, authentication/storage. Avoid mixing the Supabase Python data client and direct SQLAlchemy access for the same tables in the prototype; it creates two patterns to maintain.

4. Architecture and request flow

```
Browser / Next.js screens → FastAPI routes → service layer (workflow, deadlines, audit) → SQLAlchemy repositories/models → Supabase PostgreSQL
```

- Frontend sends HTTP requests to the FastAPI base URL; it never connects directly to PostgreSQL.
- API router validates inputs and calls a service function. Service functions enforce allowed transitions, role/scope rules, and audit requirements.
- Database transactions write the business change and corresponding audit event together when both must succeed or fail as one unit.
- Dashboard responses aggregate persisted records. P0 may use seeded synthetic data behind the same response shapes.
- A notification is a record/event in the system; email/SMS delivery is a separate integration and is not assumed for P0.

Recommended backend layers

Layer	Responsibilities	Keep out
api/	HTTP routes, dependency injection, request/response schemas, status codes.	Complex SQL or workflow decisions.
services/	Transition rules, assignment checks, deadlines, audit creation, orchestration.	HTTP-specific details where avoidable.
models.py / db models	Tables, relationships, constraints, indexes.	UI presentation logic.
schemas.py	Pydantic input/output models and enums.	Database session management.
database.py	Engine/session configuration and transaction/session dependencies.	Business rules.

5. Modules and ownership

Module	Backend responsibility	Frontend / integration touchpoint
Applications	List, detail, filters, status, stage, assigned owner, timeline.	Application list and detail screens.
Workflow	Validate configured next stage, return/revert reasons, record transitions.	Stage action buttons and confirmation dialogs.
Dashboard	Counts, stage breakdown, ageing buckets, workload summaries.	KPI cards, charts, filters.
EVC	Queue, assignment, scheduling metadata, completion and due dates.	EVC queue/calendar and status panel.
Compliance	Undertakings, evidence metadata, document expiry, review state.	Compliance tracker, evidence review, expiry badges.
Notifications	Persist in-app alerts, read/unread state, severity and due date.	Notification centre and alert banners.
Audit	Append-only event records for important actions.	Application timeline and audit panel.
Auth/permissions	Identify actor and enforce server-side role/scope.	Role-aware controls; UI visibility is not security.

Backend developer owns the API, data model, service rules and tests. The integration developer owns connecting screens to the agreed API contract, handling loading/error/empty states, and reporting contract mismatches. Frontend developers own visual layout and interaction states. Agree on response schemas before parallel implementation.

6. Data model

Use a compact relational model. The exact schema can evolve, but identifiers, enums, timestamps, and ownership semantics should be agreed early.

Entity	Important fields (illustrative)	Notes
applications	id, application_number, institution_id, region, application_type, current_stage, operational_status, assigned_user_id, submitted_at, updated_at	Main record; keep current status for fast queries.
institutions	id, name, institution_code, region	Synthetic institution records for demo.
workflow_routes	id, route_code, version, active, definition_json	Configured stage sequence/conditions; validate against APH.
workflow_events	id, application_id, from_stage, to_stage, action, reason, actor_id, created_at	Timeline and transition evidence.
assignments	id, entity_type, entity_id, assignee_id, assigned_by, created_at	Assignment history; current owner can also be cached on entity.
evc_events	id, application_id, status, scheduled_at, completed_at, assigned_team, notes	Scheduling and completion tracking; no automated inspection decision.
undertakings	id, application_id, title, due_date, status, submitted_at, verified_by, verified_at	Time-bound commitment and verification state.
documents	id, application_id, document_type, storage_key, valid_from, expires_on, review_status	Store private storage key/metadata, not public file URL.
notifications	id, recipient_id, entity_type, entity_id, severity, message, due_at, read_at, created_at	In-app notification record.
audit_events	id, actor_id, action, entity_type, entity_id, before_json, after_json, reason, created_at	Append-only application-level audit trail.

Entity	Important fields (illustrative)	Notes
users / roles	id, display_name, role, region_scope, active	For real auth, map to a verified identity; never trust a client-supplied actor ID.

Recommended enums include application operational status (e.g. DRAFT, SUBMITTED, IN_REVIEW, RETURNED, COMPLETED), workflow stage (e.g. SCRUTINY, EVC, SAC_SHC, FINAL_REVIEW), EVC status, evidence review status, and notification severity. Treat these as proposed internal names—not official AICTE labels. Keep stage progression in configuration/service rules rather than assuming every application follows the same path.

7. API design and key contracts

Base path: /api/v1. These are proposed application endpoints, not existing AICTE endpoints. Return consistent JSON and use HTTP status codes meaningfully. Add pagination/filtering to list endpoints as soon as the UI needs them.

Method + path	Scope	Purpose
GET /dashboard/summary	P0	KPI counts, stage distribution, ageing and workload aggregates.
GET /applications	P0	Paginated/filterable application list.
GET /applications/{id}	P0	Application detail, current stage and summary fields.
GET /applications/{id}/timeline	P0	Workflow/audit timeline.
POST /applications/{id}/return	P0	Return for clarification with mandatory reason and requested items.
POST /applications/{id}/transition	P0	Attempt a configured, authorised stage transition.
GET /evc	P0	EVC queue, status and due/scheduled dates.
GET /compliance	P0	Undertakings, evidence status and upcoming expiries.
GET /notifications	P0	Recipient-specific alerts; synthetic records acceptable for P0.
GET /audit/{entity_id}	P0	Audit events for an entity, subject to access checks.
POST /applications/{id}/resubmit	P1	Resubmit after clarifications/evidence are addressed.
POST /evc/{id}/schedule	P1	Schedule/reschedule an EVC event with validation.
POST /evidence/{id}/review	P1	Record an authorised evidence review outcome and reason.
POST /assignments	P1	Assign/reassign an entity to an authorised officer/team.
GET /health	P0	Basic service health; no secrets or internal diagnostics.

Example: return an application

```
POST /api/v1/applications/APP-1001/return
```

```
{ "reason": "Required lease evidence is missing", "requested_items": ["Updated lease evidence"] }
```

```
{ "application_id": "APP-1001", "status": "RETURNED", "message": "Application returned for clarification",
"audit_event_created": true }
```

Validation: reason must be non-empty and bounded; caller must be authorised; current state must permit return; the response must not be successful if the status update and audit event fail. In P1, commit the status change, workflow event and audit

event in one database transaction.

Example: application list response

```
{ "items": [{ "id": "uuid", "application_number": "APP-1001", "institution_name": "Example Institute of
Technology", "region": "Eastern Region", "current_stage": "SCRUTINY", "operational_status": "IN_REVIEW",
"assigned_to": "Officer A", "updated_at": "2026-10-29T12:30:00Z" }], "page": 1, "page_size": 20, "total": 1 }
```

API contract conventions

- Use UUIDs internally and a separate application_number for humans. Document whether route IDs accept UUIDs or public application numbers; do not mix accidentally.
- Use 400 for malformed input, 401 for unauthenticated requests, 403 for insufficient permissions, 404 for inaccessible/not-found records, 409 for invalid state transitions, and 422 for schema validation where FastAPI applies it.
- For list endpoints, define filter names and enum values centrally; provide deterministic sorting and pagination.
- Define shared response schemas in the API docs and let the integration teammate use FastAPI /docs and the OpenAPI schema.

8. Workflow engine and state transitions

Implement a small explicit transition service. The frontend requests an action; the backend decides whether it is permitted based on current state, route configuration, actor permissions, and prerequisite completion. The UI must never decide the final status by itself.

```
transition_application(application, action, actor, reason=None):
1. Load application and route configuration.
2. Confirm actor and scope are authorised.
3. Validate current state, requested action, prerequisites and destination.
4. Validate mandatory reason/evidence for return/revert actions.
5. In one transaction: update current state, insert workflow event, insert audit event.
6. Commit and return the canonical updated record.
```

Rules

- A route is selected/configured for the application type and applicable rule version; do not assume one universal linear path.
- A transition cannot skip a required stage or prerequisite unless a validated route explicitly permits it.
- Return/revert requires a reason, records who initiated it and when, and identifies requested clarification items where applicable.
- Resubmission should preserve prior history and move to the appropriate configured stage, not erase the return event.
- Human approvals, committee outcomes, inspections and verification remain explicit authorised actions.
- Invalid transitions return a conflict response and create no partial status change.

P0 can simulate a small set of configured routes with synthetic data, but still show the transition validation in the backend. P1 should add route versioning, stronger permission checks, transactional persistence, and tests for each permitted/forbidden transition.

9. Authentication, authorization, audit, and data protection

- Use a clear P0 demo mode with synthetic accounts/roles only if the UI needs role switching. Label it demo-only; do not treat a client-supplied role as secure identity.
- P1: authenticate users and derive actor identity from a verified token/session. Enforce permissions and region/record scope in backend dependencies/service layer.
- Use least-privilege database credentials. Keep database URLs, passwords, JWT secrets and service-role keys server-side in environment variables.
- If Supabase Row Level Security is enabled, define and test policies deliberately. Never expose a privileged service-role key in browser code.
- Store sensitive evidence in private storage. Check authorization before issuing access links; record uploads, downloads where required, review actions and metadata changes.
- Audit events should include actor, action, entity, timestamp, reason, and relevant before/after values. Treat audit history as append-only through the application API.

- Configure CORS for the known frontend origin(s), not a permissive wildcard in a production deployment. Avoid logging sensitive document contents or secrets.

The hackathon's synthetic data should not contain real student, institution, officer, identity, or confidential approval records. Production deployment requires a separate privacy, security, retention, backup, access-review and incident-response assessment.

10. Deadlines, alerts, and escalation

Trigger	P0 behaviour	P1/P2 behaviour
Stage ageing / SLA	Display due date and ageing badge using seeded timestamps.	Compute from validated configurable policy and business calendar.
EVC scheduling/completion	Show queue, schedule/status fields and a demo reminder.	Persist schedule, detect overdue items, notify responsible users.
Undertaking deadline	List due/overdue items and status.	Scheduled checks create idempotent alerts and escalation events.
Document expiry	Show expiry date and upcoming/expired badges.	Configurable reminder windows, review/renewal flow and notification history.
Escalation	Show simulated escalation badge/event.	Persist escalation level, recipient, timestamp and acknowledgement.

- Use UTC timestamps for events; render in local timezone. For expiry dates, agree whether the rule means end-of-day local time or a date-only comparison.
- Make alert creation idempotent: rerunning a deadline scan must not create duplicate alerts for the same trigger/window.
- Keep notification generation separate from email/SMS delivery. P0 can use in-app notifications only.
- Do not invent official SLA thresholds. Use clearly labelled demo thresholds until confirmed from authorised rules.

11. Frontend-to-backend integration contract

How the integration teammate should work

- Use the agreed API base URL from an environment variable, e.g. NEXT_PUBLIC_API_BASE_URL for the public base URL only.
- Create one typed API client module; do not scatter raw fetch calls and hard-coded endpoint strings across components.
- For each screen, document endpoint, request parameters, response shape, loading state, empty state, validation errors, and retry behaviour.
- Use FastAPI /docs and OpenAPI to inspect live schemas. If an endpoint is not ready, agree on a mock response that matches the final schema rather than inventing a separate shape.
- Show backend validation errors to the user in a useful form; do not silently change a status locally after a failed POST.
- Agree early on enums and field names. If a contract changes, notify frontend and integration teammates in the same pull request/issue.

Minimum contract sheet

Screen	Endpoint(s)	Key fields
Dashboard	GET /dashboard/summary	total_applications, by_stage, overdue_count, upcoming_expiries, workload
Application list	GET /applications	application_number, institution_name, region, current_stage, operational_status, assigned_to, updated_at
Application detail	GET /applications/{id}; GET /applications/{id}/timeline	route, current status, prerequisites, events, return reason, due date
EVC queue	GET /evc	application_number, status, scheduled_at, due_at, assigned_team
Compliance	GET /compliance	item_type, title, due_date/expires_on, status, days_remaining
Notifications	GET /notifications	severity, message, entity link, due_at, read_at

12. Environment setup and deployment

Backend developer setup

```
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1
# macOS/Linux
source .venv/bin/activate
pip install fastapi uvicorn sqlalchemy "psycopg[binary]" pydantic-settings pytest httpx
uvicorn app.main:app --reload
```

Create backend/.env locally with DATABASE_URL and any required settings. Keep it out of Git. Commit a .env.example containing placeholder values only. Confirm whether the hosted database connection requires a direct or pooled URL and use the provider's current connection guidance. For a hackathon, use the simplest stable connection option that works with the chosen deployment.

Frontend / integration setup

```
NEXT_PUBLIC_API_BASE_URL=http://127.0.0.1:8000
# Frontend starts using its normal Next.js dev command.
# API docs: http://127.0.0.1:8000/docs
```

- Local development: frontend and FastAPI run as separate processes; database is hosted in Supabase.
- Deployment: frontend may be hosted on Vercel; FastAPI needs a Python-compatible host. Confirm the host supports long-running HTTP requests, environment variables, and the selected Python runtime.
- Use separate development/demo database credentials; do not point experimental migrations at production data.
- Keep deployment optional until the local end-to-end demo works. A reliable local demo is better than a broken public URL.

13. Build sequence: P0 / P1 / P2

Priority	Scope	Definition of done
P0 — submission prototype	Polished screens, synthetic seeded records, list/detail, dashboard, visible route stages, return/transition interactions, EVC queue, compliance/expiry, notifications and audit timeline.	A judge can follow one application end-to-end; status and alert states are understandable; demo data is labelled; no dead-end buttons.
P1 — functional MVP	Persisted PostgreSQL records, authentication, server-side role/scope checks, real transition validation, transactional audit, assignment/resubmission, evidence review, idempotent deadline checks.	Actions survive refresh; invalid transitions are rejected; audit is consistent; tests cover core workflow and permissions.
P2 — production readiness	APH-validated rules, authorised system integrations, secure file storage, reliable scheduler/delivery, monitoring, backups, retention and security testing.	Operational, legal, security and integration owners sign off; no claim of production readiness without validation.

Backend build order for the sole backend developer

- Step 1 — service skeleton: create FastAPI app, health endpoint, CORS, settings and a consistent error shape.
- Step 2 — unblock integration: implement GET /api/v1/applications and GET /api/v1/applications/{id} with stable Pydantic schemas and seed data. If database setup slows progress, use an in-memory repository behind the same service interface temporarily.
- Step 3 — workflow core: implement allowed-transition checks, return reason validation, timeline events and audit service.
- Step 4 — dashboard/queues: implement summary, EVC and compliance read endpoints using shared synthetic or persisted records.
- Step 5 — persistence: add PostgreSQL models, migrations, transaction boundaries and seed script; avoid maintaining two divergent data stores.
- Step 6 — hardening: permissions, notification persistence, deadline scanning and automated tests, according to time available.

Do not block the integration teammate on the full database schema. Publish stable response models and seed responses early; swap the repository implementation underneath without changing the contract.

14. Four-person work plan and Git workflow

Person	Primary ownership	First deliverable
Frontend developer A	Dashboard, navigation, global layout, charts and filters.	Dashboard shell with loading/empty/error states and agreed fields.
Frontend developer B	Application list/detail, workflow dialogs, EVC and compliance screens.	Screens wired to mock data that matches API schemas.
Backend developer (user)	FastAPI, schemas, service rules, DB, audit, tests and API docs.	Working API + list/detail endpoints + synthetic seed data.
Integration developer	API client, endpoint wiring, data mapping, integration checks and issue triage.	Typed API client; connect list/detail first; track missing fields and failures.

Coordination rules

- Create one shared API contract sheet and treat schema changes as coordinated changes.
- Use small branches and pull requests; avoid multiple people editing the same backend files.
- Protect a working demo branch. Merge only changes that start, build and preserve the demo path.
- Agree on the route/status enums before implementing action buttons or dashboard aggregates.
- Hold short integration checkpoints: after list/detail, after transition/return, and before final demo.

- Freeze new features before submission; reserve the final block for integration, seed data, and rehearsing the demo.

Suggested order for the first working slice: FastAPI health → seeded applications list → application detail → frontend wiring → transition/return → audit timeline → dashboard → EVC/compliance alerts.

15. Testing and acceptance strategy

Test layer	Minimum checks
Schema/unit	Missing/blank return reason rejected; unknown enum rejected; date fields parse correctly.
Workflow service	Valid transition succeeds; forbidden transition returns conflict; missing prerequisite blocks progression; return requires reason.
Audit/transaction	Successful mutation creates an event; failed mutation creates no partial state or misleading success event.
Permissions	Unauthorised role/scope cannot transition, assign, review evidence, or read restricted records.
Deadline logic	Upcoming/overdue boundaries correct; repeated scan does not duplicate same alert; expiry date semantics tested.
API integration	List/detail/summary return documented schemas; filters and pagination behave consistently.
UI integration	Loading, empty, error, success and validation states; failed POST does not optimistically show a false success.
Demo regression	Seed reset works; known demo application follows the intended route; refresh does not break P0 narrative.

Acceptance criteria for a judge-ready P0

- A user can locate an application and see institution, region, stage, status, owner and age/due-date indicators.
- The detail view shows route progress and prior actions.
- A return action requires a reason and visibly updates the application/timeline in the demo.
- Dashboard counts and EVC/compliance queues agree with the seeded dataset.
- Upcoming and expired document/undertaking items are visible, with a clear reminder/escalation state.
- A user can see an audit/timeline entry for significant demo actions.
- No control claims a statutory approval/rejection has been made automatically.

16. Risks and decisions to validate

Risk / unknown	Mitigation / required decision
APH route/timeline ambiguity	Validate actual routes, prerequisites, deadlines, terminology and competent authorities against the applicable APH before production.
Too much scope for one backend developer	Lock P0 to a single end-to-end path plus representative dashboard/queue data; defer complex integrations.
Frontend/backend contract drift	Publish schemas early, use /docs, centralise enums, and have one integration owner.
Database setup delays	Use a repository abstraction and synthetic seed data; do not change API shapes when persistence is added.
Fake alerts mistaken for live notifications	Label P0 alerts as demo/simulated and state which delivery channels are not integrated.
Security shortcuts carried into production	Keep demo mode explicit; never expose secrets; require auth and access-policy testing before real data.
Public hosting instability	Rehearse on the exact demo machine and keep a known-good local fallback.

Open decisions: (1) which APH/version applies to each application type; (2) official deadline/SLA rules and holiday calendar; (3) required roles and region boundaries; (4) whether approved identity, document repository, email/SMS or AICTE systems expose authorised APIs; (5) retention and audit requirements; (6) deployment and data-residency requirements.

17. End-to-end demo checklist

- Reset to known synthetic seed data and verify dashboard totals.
- Open an application in scrutiny and show the configured route and pending/ageing information.
- Attempt a valid transition and show the updated stage and timeline.
- Return another application for clarification; show required reason, requested items and audit entry.
- Show EVC queue, a scheduled/due item and an overdue/escalation example.
- Show an undertaking approaching its due date and a document that is expired or nearing expiry.
- Show notifications and entity audit history.
- Demonstrate a blocked invalid transition to prove the backend enforces rules.
- State clearly what is simulated in P0 and what is planned for P1/P2.

Appendix A — Suggested repository structure

```
project-root/ ■■■ frontend/ # Next.js + TypeScript ■ ■■■ app/ ■ ■■■ components/ ■ ■■■ lib/api-client.ts ■ ■■■ types/api.ts ■■■ backend/ ■ ■■■ app/ ■ ■ ■■■ main.py ■ ■ ■■■ settings.py ■ ■ ■■■ database.py ■ ■ ■■■ models.py ■ ■ ■■■ schemas.py ■ ■ ■■■ api/ ■ ■ ■ ■■■ applications.py ■ ■ ■ ■■■ dashboard.py ■ ■ ■ ■■■ evc.py ■ ■ ■ ■■■ compliance.py ■ ■ ■ ■■■ notifications.py ■ ■ ■ ■■■ audit.py ■ ■ ■■■ services/ ■ ■ ■■■ workflow.py ■ ■ ■■■ audit.py ■ ■ ■■■ deadlines.py ■ ■■■ tests/ ■ ■■■ .env.example ■ ■■■ requirements.txt ■■■ docs/ ■ ■■■ api-contracts.md ■■■ README.md
```

Keep modules small enough for the single backend developer to own. Do not add empty abstractions or directories until the first feature needs them.

Appendix B — Initial backend dependencies

```
fastapi uvicorn[standard] sqlalchemy psycopg[binary] pydantic-settings pytest httpx
```

Add Alembic for migrations when persistence/schema changes begin. Pin tested versions before submission/deployment. Add authentication and storage libraries only when the team selects a specific implementation; do not copy credentials or production secrets into the repository.

Appendix C — Proposed initial Pydantic schema

```
from pydantic import BaseModel, Field from datetime import datetime from enum import Enum class
ApplicationStatus(str, Enum): SUBMITTED = "SUBMITTED" IN_REVIEW = "IN_REVIEW" RETURNED = "RETURNED" COMPLETED =
"COMPLETED" class ReturnApplicationRequest(BaseModel): reason: str = Field(min_length=3, max_length=1000)
requested_items: list[str] = Field(default_factory=list) class ApplicationSummary(BaseModel): id: str
application_number: str institution_name: str current_stage: str operational_status: ApplicationStatus
assigned_to: str | None = None updated_at: datetime
```

This is a starting example only. Expand enums and schemas in collaboration with frontend/integration teammates; do not use arbitrary strings for important states once the workflow is implemented.

Reference notes

The architecture choices in this version are proposed engineering decisions for the stated team and hackathon timeline. Use the current FastAPI documentation for OpenAPI-generated API documentation and the current Supabase documentation for connection methods and Row Level Security. Confirm provider-specific deployment, pooling, auth and storage details against the selected plan and environment before launch.

Regulatory routes, eligibility rules, statutory timelines and authority permissions must be confirmed from the applicable AICTE Approval Process Handbook and authorised directions. This TRD does not itself establish those rules.